

SID4: 1421

PORT_BASE: 8521

PREFIX: s1421

SEED: 1421

VERIFY_SEED: 2614221

DOMAIN_ID: 5

Hardware: Apple M2 Pro, 16GB

Local model used: qwen3:8b

Tagged commit hash: 14e0c340fcf5356f7db9e0661f49b5da5871f291

Part 1 – HTML & CCS

1.

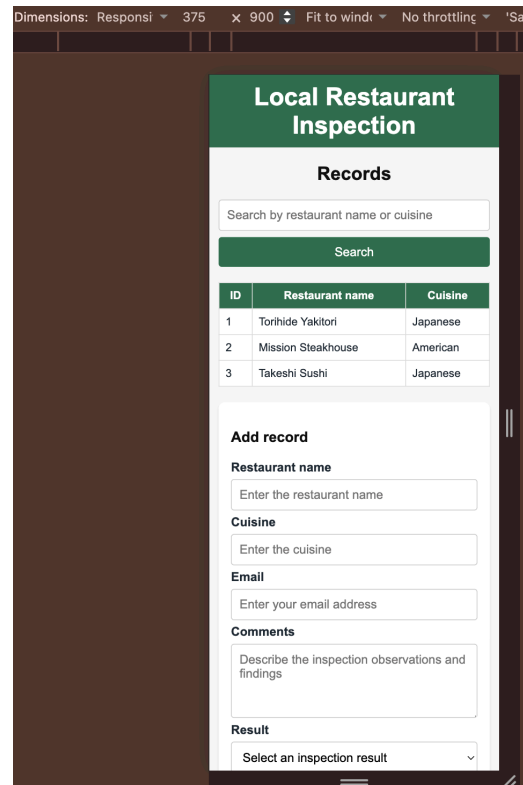
```
@media (max-width: 375px) {
  .search-row {
    flex-direction: column;
    padding: 0 12px;
  }

  .search-row button {
    width: 100%;
  }

  .table-wrap {
    width: calc(100% - 24px);
  }

  .record-table th,
  .record-table td {
    padding: 8px;
    font-size: 0.9rem;
  }

  .actions-container {
    margin: 16px 12px;
    padding: 16px;
  }
}
```

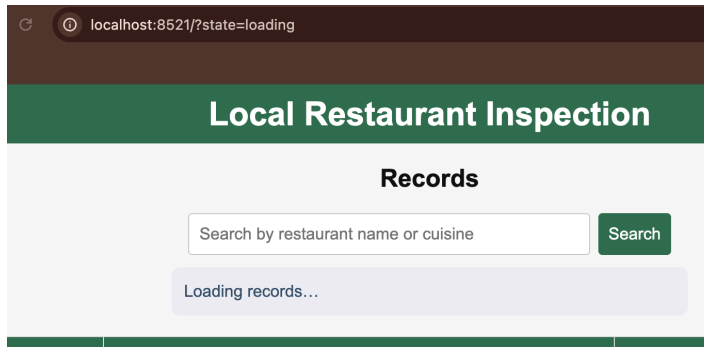


2.

Loading:

```
<p id="loadingMessage" class="status status-loading" hidden>
  Loading records...
</p>
```

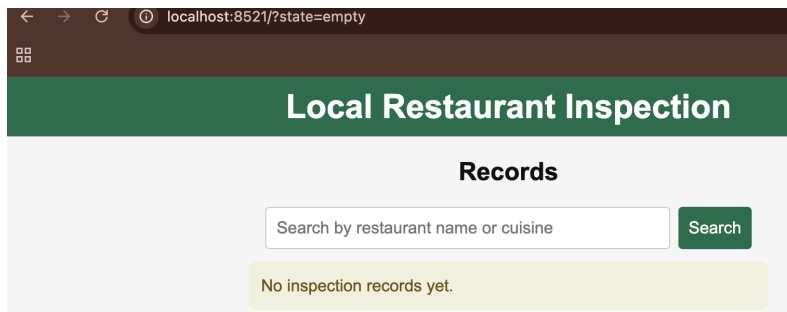
```
const showLoading = () => {
  hideStatusMessages();
  recordList.replaceChildren();
  loadingMessage.hidden = false;
};
```



Empty:

```
<p id="emptyMessage" class="status status-empty" hidden>
  No inspection records yet.
</p>
```

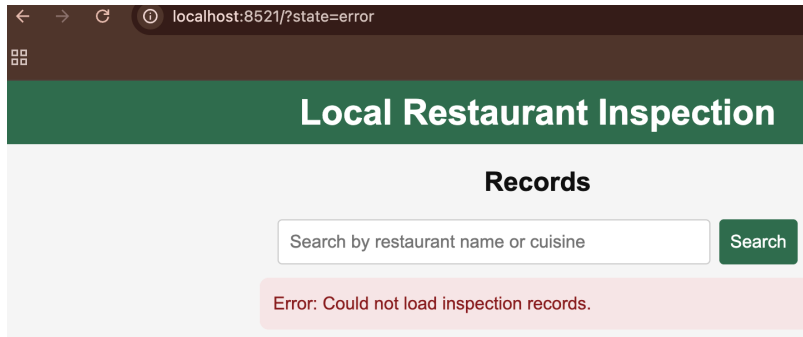
```
const showEmpty = () => {
  hideStatusMessages();
  recordList.replaceChildren();
  emptyMessage.hidden = false;
};
```



Error:

```
<p id="errorMessage" class="status status-error" hidden></p>
```

```
const showError = (message) => {
  hideStatusMessages();
  recordList.replaceChildren();
  errorMessage.textContent = message;
  errorMessage.hidden = false;
};
```



Part 2 – FastAPI

1.

Codes:

```
<div class="actions-container">
  <h3>Add record</h3>
  <form id="inspectionForm" onsubmit="createRecord(); return false;">
```

```
try {
  const response = await fetch("/api/records", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      restaurantName: updatedObject.restaurantName,
      cuisine: updatedObject.cuisine,
      Email: updatedObject.Email,
      Comments: updatedObject.Comments,
      Result: updatedObject.Result,
      termsAccepted: updatedObject.termsAccepted,
      submissionDate: updatedObject.submissionDate,
    }),
  });
}
```

```
@app.post("/api/records", response_model=InspectionRecord, status_code=201)
def create_record_api(payload: InspectionCreate):
    return add_record(payload)
```

Before adding a new record:

Local Restaurant Inspection

Records

Search by restaurant name or cuisine

Search

ID	Restaurant name	
1	Torihide Yakitori	Japanese
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese

Add record

Adding a new record:

1	Torihide Yakitori	Japanese
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese

Add record

Restaurant name

Eric Fried Rice

Cuisine

Chinese

Email

eric.e.zhao@sjsu.edu

Comments

The restaurant met compliance in all aspects. No discrepancy was found.

Result

Excellent Pass

☒ I agree to the terms and conditions.

Add record

After adding the new record:

ID	Restaurant name	Cuisine
1	Torihide Yakitori	Japanese
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese
4	Eric Fried Rice	Chinese

2.

Codes:

```
<div class="actions-container">
  <h3>Update record 1</h3>
  <form id="updateForm" onsubmit="updateRecord(); return false;">
    <label for="updateRestaurantName">Restaurant name</label>
    <input
      id="updateRestaurantName"
      name="restaurantName"
      type="text"
      placeholder="New restaurant name"
      required
    />
    <label for="updateCuisine">Cuisine</label>
    <input
      id="updateCuisine"
      name="cuisine"
      type="text"
      placeholder="New cuisine"
      required
    />
    <button type="button" onclick="updateRecord()">Update record 1</button>
  </form>
</div>
```

```
async function updateRecord() {
  if (!updateForm.reportValidity()) {
    return;
  }

  const restaurantName = document.querySelector("#updateRestaurantName").value.trim();
  const cuisine = document.querySelector("#updateCuisine").value.trim();

  try {
    const response = await fetch("/api/records/1", {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ restaurantName, cuisine }),
    });

    if (!response.ok) {
      throw new Error("Could not update record 1.");
    }

    updateForm.reset();
    await loadRecords(searchQuery.value.trim());
  } catch (error) {
    showError(error.message);
  }
}
```

```
@app.put("/api/records/1", response_model=InspectionRecord)
def update_record_one_api(payload: InspectionUpdate):
    restaurant_name = payload.restaurantName.strip()
    cuisine = payload.cuisine.strip()
    if not restaurant_name or not cuisine:
        raise HTTPException(status_code=400, detail="Restaurant name and cuisine are required")

    record = next((item for item in records if item.id == 1), None)
    if record is None:
        raise HTTPException(status_code=404, detail="Record 1 was not found")

    record.restaurantName = restaurant_name
    record.cuisine = cuisine
    return record
```

Before updating record ID1:

ID	Restaurant name	Cuisine
1	Torihide Yakitori	Japanese
2	Mission Steakhouse	American

Updating record ID1:

Update record 1

Restaurant name

Cuisine

Update record 1

After updating record ID1:

ID	Restaurant name	Cuisine
1	l'arc en ciel	French
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese

3.

Codes:

```
<div class="actions-container">
  <h3>Delete highest ID</h3>
  <form id="deleteForm" onsubmit="deleteRecord(); return false;">
    <button type="button" onclick="deleteRecord()" class="danger">Delete highest ID</button>
  </form>
</div>
```

```
async function deleteRecord() {
  try {
    const response = await fetch("/api/records/highest", { method: "DELETE" });
    if (!response.ok) {
      throw new Error("Could not delete the highest ID.");
    }
    await loadRecords(searchQuery.value.trim());
  } catch (error) {
    showError(error.message);
  }
}
```

```
@app.delete("/api/records/highest", status_code=204)
def delete_highest_record_api():
    if not records:
        raise HTTPException(status_code=404, detail="No records to delete")
    highest_id = max(record.id for record in records)
    records[:] = [record for record in records if record.id != highest_id]
    return Response(status_code=204)
```

Before deleting the record with the highest ID:

ID	Restaurant name	Cuisine
1	l'arc en ciel	French
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese
4	Eric Fried Rice	Chinese

Deleting the record with the highest ID:

Delete highest ID

Delete highest ID

After deleting the record with the highest ID:

ID	Restaurant name	Cuisine
1	l'arc en ciel	French
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese

4.

Codes:

```
<form id="searchForm" class="search-row" onsubmit="searchRecords(); return false;">
  <label class="sr-only" for="searchQuery">Search</label>
  <input
    id="searchQuery"
    name="q"
    type="search"
    placeholder="Search by restaurant name or cuisine"
  />
  <button type="button" onclick="searchRecords()">Search</button>
</form>
```

```
function searchRecords() {  
  loadRecords(searchQuery.value.trim());  
}
```

```
@app.get("/api/records", response_model=list[InspectionRecord])  
def list_records(  
    response: Response,  
    q: str = Query(default=""),  
):  
    response.headers["Cache-Control"] = "no-store"  
    return matching_records(q)
```

```
def matching_records(query: str) -> list[InspectionRecord]:  
    needle = query.strip().lower()  
    if not needle:  
        return list[InspectionRecord](records)  
    return [  
        record  
        for record in records  
        if needle in record.restaurantName.lower() or needle in record.cuisine.lower()  
    ]
```

Before searching:

ID	Restaurant name	Cuisine
1	L'arc en ciel	French
2	Mission Steakhouse	American
3	Takeshi Sushi	Japanese

After searching:

By primary field:

ID	Restaurant name	Cuisine
2	Mission Steakhouse	American

By secondary field:

ID	Restaurant name	Cuisine
2	Mission Steakhouse	American

Part3 – Stateful Agent Graph

state.py

```
1  from typing import Any, Dict, TypedDict
2
3  class AgentState(TypedDict):
4      title: str
5      content: str
6      email: str
7      strict: bool
8      task: str
9      llm: Any
10     planner_proposal: Dict[str, Any]
11     reviewer_feedback: Dict[str, Any]
12     turn_count: int
```

planner node:

```
def planner_node(state: AgentState) -> Dict[str, Any]:
    print("[Planner] Node activated")
    llm = state["llm"]
    feedback = state.get("reviewer_feedback") or {}

    system_text = (
        "You are Planner. Propose exactly 3 distinct, topical tags "
        "(prefer multi-word phrases) and a one-line summary for the given title and content. "
        "Return only JSON."
    )
    human = (
        f"Task: {state['task']}\n"
        f"Title: {state['title']}\n"
        f"Content: {state['content']}\n"
        f"Reviewer feedback: {json.dumps(feedback, ensure_ascii=False)}\n"
        "Return ONLY one JSON object with keys: thought, message, "
        "data.tags (exactly 3 tags), data.summary (<=25 words), data.issues."
    )

    proposal: Dict[str, Any] = {}
    try:
        t0 = time.time()
        response = llm.complete([
            {"role": "system", "content": system_text},
            {"role": "user", "content": human},
        ])
        t1 = time.time()
        print(f"[Planner] latency_ms={int((t1 - t0) * 1000)}")
        proposal = parse_and_coerce(
            response.content,
            state["title"],
            state["content"],
            state["strict"],
        )
    except Exception as err:
        print(f"[Planner] ERROR: {err}")
        proposal = {"error": str(err)}

    print("[Planner] complete")
    return {
        "planner_proposal": proposal,
        "reviewer_feedback": {},
    }
```

reviewer_node:

```
def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print("[Reviewer] Node activated")
    llm = state["llm"]
    proposal = state.get("planner_proposal") or {}

    system_text = (
        "You are Reviewer. Validate: tags topical and not generic; "
        "summary <= 25 words; no code or markdown. "
        "If issues, list them in data.issues; otherwise echo cleaned tags/summary. "
        "Return only JSON."
    )
    human = (
        f"Task: {state['task']}\n"
        f"Title: {state['title']}\n"
        f"Content: {state['content']}\n"
        f"Planner proposal: {json.dumps(proposal, ensure_ascii=False)}\n"
        "Return ONLY one JSON object with keys: thought, message, "
        "data.tags, data.summary, data.issues."
    )

    feedback: Dict[str, Any] = {}
    try:
        t0 = time.time()
        response = llm.complete([
            {"role": "system", "content": system_text},
            {"role": "user", "content": human},
        ])
        t1 = time.time()
        print(f"[Reviewer] latency_ms={int((t1 - t0) * 1000)}")
        feedback = parse_and_coerce(
            response.content,
            state["title"],
            state["content"],
            state["strict"],
        )
    except Exception as err:
        print(f"[Reviewer] ERROR: {err}")
        feedback = {"error": str(err)}

    print("[Reviewer] complete")
    return {"reviewer_feedback": feedback}
```

supervisor_node:

```
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    print("[Supervisor] Node activated")
    turn_count = int(state.get("turn_count") or 0) + 1
    print(f"[Supervisor] turn_count={turn_count}")
    return {"turn_count": turn_count}
```

router_logic:

```
def router_logic(state: AgentState) -> Literal["planner", "reviewer", "END"]:
    max_turns = int(state.get("max_turns") or 10)
    turn_count = int(state.get("turn_count") or 0)

    if turn_count > max_turns:
        return "END"

    proposal = state.get("planner_proposal") or {}
    if not proposal:
        return "planner"

    feedback = state.get("reviewer_feedback") or {}
    if not feedback:
        return "reviewer"

    issues = []
    data = feedback.get("data") if isinstance(feedback, dict) else {}
    if isinstance(data, dict):
        issues = data.get("issues") or []

    if issues:
        return "planner"
    return "END"
```

workflow:

```
def build_workflow():
    workflow = StateGraph(AgentState)

    workflow.add_node("supervisor", supervisor_node)
    workflow.add_node("planner", planner_node)
    workflow.add_node("reviewer", reviewer_node)

    workflow.set_entry_point("supervisor")

    workflow.add_conditional_edges(
        "supervisor",
        router_logic,
        {
            "planner": "planner",
            "reviewer": "reviewer",
            "END": END,
        },
    )

    workflow.add_edge("planner", "supervisor")
    workflow.add_edge("reviewer", "supervisor")

    return workflow.compile()
```

```

47 def main():
48     llm = OllamaModelClient(output_format="json")
49     title = "Yakitori Restaurant Inspections"
50     content = (
51         "Observed multiple live flies landing on clean food-contact storage racks "
52         "and food prep counters; small rodent droppings noted near the dry storage "
53         "shelving baseboards."
54     )
55     email = "eric.e.zhao@sjsu.edu"
56     initial_state: AgentState = {
57         "title": title,
58         "content": content,
59         "email": email,
60         "strict": False,
61         "task": (
62             f'Given the title "{title}" and content "{content}", '
63             "produce exactly 3 topical tags and a one-sentence summary. "
64             f"Email is {email}."
65         ),
66         "llm": llm,
67         "planner_proposal": {},
68         "reviewer_feedback": {},
69         "turn_count": 0,
70     }
71     app = build_workflow()
72     for event in app.stream(initial_state):
73         print(event)
74
75
76 if __name__ == "__main__":
77     main()

```

```

code -- zsh -- 126x39

Last login: Sun Sep 13 03:00:37 on ttys000
[base] eric@Eric-Mac ~ % conda activate data260
[data260] eric@Eric-Mac ~ % cd data260/code
[data260] eric@Eric-Mac code % python -m restaurant_graph.workflow
<frozen runpy>:128: RuntimeWarning: 'restaurant_graph.workflow' found in sys.modules after import of package 'restaurant_graph', but prior to execution of 'restaurant_graph.workflow'; this may result in unpredictable behaviour
[Supervisor] Node activated
[Supervisor] turn_count=1
{'supervisor': {'turn_count': 1}}
[Planner] Node activated
[Planner] latency_ms=5467
[Planner] complete
{'planner': {'planner_proposal': {'thought': 'The content describes hygiene and pest issues in a yakitori restaurant during an inspection, requiring specific tags and a concise summary.', 'message': 'Tags and summary generated based on the inspection findings.', 'data': {'tags': ['yakitori restaurant inspections', 'yakitori restaurant', 'restaurant inspections'], 'summary': 'Inspection found live flies and rodent droppings in a yakitori restaurant, indicating hygiene and pest control problems.', 'issues': ['Live flies on food-contact surfaces', 'Rodent droppings near dry storage', 'Potential food safety and pest control violations']}, 'reviewer_feedback': {}}}}
[Supervisor] Node activated
[Supervisor] turn_count=2
{'supervisor': {'turn_count': 2}}
[Reviewer] Node activated
[Reviewer] latency_ms=4754
[Reviewer] complete
{'reviewer': {'reviewer_feedback': {'thought': 'The content describes hygiene and pest issues in a yakitori restaurant during an inspection, requiring specific tags and a concise summary.', 'message': 'Tags and summary generated based on the inspection findings.', 'data': {'tags': ['yakitori restaurant inspections', 'yakitori restaurant', 'restaurant inspections'], 'summary': 'Inspection found live flies and rodent droppings in a yakitori restaurant, indicating hygiene and pest control problems.', 'issues': ['Live flies on food-contact surfaces', 'Rodent droppings near dry storage', 'Potential food safety and pest control

```

Part4 – Output Schema and Loop Safety

1.

```
class PlannerOutput(BaseModel):
    tags: list[str] = Field(min_length=3, max_length=3)
    summary: str

    @field_validator("tags")
    @classmethod
    def tag_length(cls, tags: list[str]) -> list[str]:
        cleaned = []
        for tag in tags:
            text = str(tag).strip()
            if len(text) < 3 or len(text) > 30:
                raise ValueError("each tag must be 3-30 characters")
            cleaned.append(text)
        return cleaned

    @field_validator("summary")
    @classmethod
    def summary_word_limit(cls, summary: str) -> str:
        text = str(summary).strip()
        if not text:
            raise ValueError("summary is empty")
        if len(text.split()) > 25:
            raise ValueError("summary must be at most 25 words")
        return text
```

```
def validate_planner_output(text: str) -> Tuple[Dict[str, Any] | None, str]:
    parsed = parse_planner_json(text)
    try:
        output = PlannerOutput.model_validate(
            {"tags": parsed.get("tags"), "summary": parsed.get("summary")}
        )
    except ValidationError as err:
        return None, err.errors()[0].get("msg") if err.errors() else str(err)

    proposal = {
        "thought": str(parsed.get("thought") or ""),
        "message": str(parsed.get("message") or ""),
        "data": {
            "tags": output.tags,
            "summary": output.summary,
            "issues": [],
        },
    },
    return proposal, ""
```

```
(data260) eric@Eric-Mac code % python - << 'PY'
import json, sys
sys.path[:0] = ["..", ".."]
from src.model_client import OllamaModelClient
from restaurant_graph.workflow import make_state, run_graph, classify_schema_run
case = json.loads(open("../reports/hw02/cases/schema_input.json").read())
llm = OllamaModelClient(output_format="json", temperature=0.0)
state = make_state(llm, case["title"], case["content"], case["email"], max_turns=2, schema_only=True)
final, ms = run_graph(state)
data = (final.get("planner_proposal") or {}).get("data") or {}
print("outcome:", classify_schema_run(final))
print("tags:", data.get("tags"))
print("n_tags:", len(data.get("tags") or []))
print("summary:", data.get("summary"))
print("n_words:", len(str(data.get("summary") or "").split()))
print("latency_ms:", ms)
py

[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=3392
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
outcome: valid_first_attempt
tags: ['food safety issues', 'pest infestation', 'hygiene violations']
n_tags: 3
summary: Inspection found live flies and rodent droppings in food storage areas, indicating hygiene and pest control problems.
n_words: 17
latency_ms: 3396
(data260) eric@Eric-Mac code %
```

2.

```
if error or not proposal:
    if attempts >= max_turns:
        return "END"
    return "planner"
```

```
(data260) eric@Eric-Mac code % python - << 'PY'
import json, sys
sys.path[0] = ["..", ".."]
from src.model_client import OllamaModelClient
from restaurant_graph.workflow import make_state, run_graph, classify_schema_run
case = json.loads(open("../reports/hw02/cases/adversarial_input.json").read())
llm = OllamaModelClient(output_format="json", temperature=0.0)
state = make_state(
    llm, case["title"], case["content"], case["email"],
    max_turns=2, schema_only=True,
)
final, ms = run_graph(state)
print("outcome:", classify_schema_run(final))
print("attempts:", final.get("planner_attempts"))
print("validation_error:", final.get("validation_error"))
print("latency_ms:", ms)
PY
[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=3939
[Planner] schema error: Input should be a valid list
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
[Planner] Node activated
[Planner] latency_ms=4648
[Planner] schema error: Input should be a valid list
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=3
outcome: hit_ceiling
attempts: 2
validation_error: Input should be a valid list
latency_ms: 8593
(data260) eric@Eric-Mac code %
```

3.

```
=== schema30 run 1/30 max_turns=10 ===
[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=5458
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
{"run": 1, "outcome": "valid_first_attempt", "planner_attempts": 1, "validation_error": "", "latency_ms": 5462, "tags": ["food
safety issues", "pest infestation", "hygiene violations"], "summary": "Inspection found live flies and rodent droppings in fo
od storage areas, indicating hygiene and pest control problems."}

=== schema30 run 30/30 max_turns=10 ===
[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=3404
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
{"run": 30, "outcome": "valid_first_attempt", "planner_attempts": 1, "validation_error": "", "latency_ms": 3408, "tags": ["foo
d safety issues", "pest infestation", "hygiene violations"], "summary": "Inspection found live flies and rodent droppings in f
ood storage areas, indicating hygiene and pest control problems."}
wrote /Users/eric/data260/reports/hw02/raw/schema_runs.json
```

```
{
  "started": "2026-09-14T17:43:33.928337+00:00",
  "input": {
    "title": "Yakitori Restaurant Inspections",
    "content": "Observed multiple live flies landing on clean food-contact storage racks and food pr",
    "email": "eric.e.zhao@sjsu.edu"
  },
  "max_turns": 10,
  "summary": {
    "n": 30,
    "counts": {
      "valid_first_attempt": 30
    },
    "completion_rate": 1.0,
    "mean_latency_ms": 3446.6,
    "mean_latency_completed_ms": 3446.6
  },
  "runs": [

```

4.

```
=== ceiling2 run 20/20 max_turns=2 ===
[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=3309
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
{"run": 20, "outcome": "valid_first_attempt", "planner_attempts": 1, "validation_errors": ["pest infestation", "hygiene violations"], "summary": "Inspection found live ing hygiene and pest control problems."}
```

```
=== ceiling10 run 20/20 max_turns=10 ===
[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=3346
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
{"run": 20, "outcome": "valid_first_attempt", "planner_attempts": 1, "validation_errors": ["pest infestation", "hygiene violations"], "summary": "Inspection found live ing hygiene and pest control problems."}
wrote /Users/eric/data260/reports/hw02/raw/ceiling_runs.json
```

```
15 Source: `reports/hw02/raw/schema_runs.json`
16
17 ## Turn ceiling 2 vs 10 (20 runs each)
18
19 | Ceiling | Completion rate | Mean latency (ms) |
20 | --- | --- | --- |
21 | 2 | 1.000 | 3347.8 |
22 | 10 | 1.000 | 3365.4 |
23
24 Chosen for deployment: **2**. Both ceilings completed at the same rate. Ceiling 2 is as fast or faster and stops sooner on invalid output.
25
26 Source: `reports/hw02/raw/ceiling_runs.json`
```

5.

```
((data260) eric@Eric-Mac code % python run_hw02_part4.py --experiment adversarial

=== adversarial run 1/5 max_turns=10 ===
[Supervisor] Node activated
[Supervisor] turn_count=1
[Planner] Node activated
[Planner] latency_ms=4994
[Planner] schema error: Input should be a valid list
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=2
```

```
[Planner] Node activated
[Planner] latency_ms=4451
[Planner] schema error: Input should be a valid list
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=10
{"run": 1, "outcome": "hit_ceiling", "planner_attempts": 10, "validation_error": "Input should be a valid list", "latency_ms": 45120, "tags": null, "summary": null}
```

```
[Planner] Node activated
[Planner] latency_ms=4453
[Planner] schema error: Input should be a valid list
[Planner] complete
[Supervisor] Node activated
[Supervisor] turn_count=11
{"run": 5, "outcome": "hit_ceiling", "planner_attempts": 10, "validation_error": "Input should be a valid list", "latency_ms": null, "summary": null}
wrote /Users/eric/data260/reports/hw02/raw/adversarial_runs.json
done 2026-09-14T18:32:08.608476+00:00
wrote /Users/eric/data260/reports/hw02/METRICS.md
(data260) eric@Eric-Mac code %
```

The input tells the model to emit one-letter tags ‘a,b,c’ as a comma string and a 40-word summary, so Pydantic rejects ‘data.tags’ (not a list of 3 tags each 3–30 characters) and the graph retries until the ceiling.

Proposed fix: reject non-list tags before another LLM call, or repair tags and summary with a deterministic post-processor after the first failure.